BHAR: A System-Level Optimization Framework for Multisensor Batteryless Human Activity Recognition

Geffen Cooper, *Graduate Student Member, IEEE*, Nathan Tsao, Filippos Fotiadis, *Member, IEEE*, Ufuk Topcu, *Fellow, IEEE*, and Radu Marculescu, *Fellow, IEEE*

***Abstract*—The emergence of batteryless sensors offers a sustainable path toward ubiquitous sensing. However, unlike battery-powered devices, batteryless sensors only operate once they have harvested sufficient energy from the environment. This disrupts data availability and timely predictions for time series tasks like human activity recognition (HAR). Existing machine learning (ML) pipelines are ill-equipped for this uncertainty in data acquisition, as they typically assume synchronized and continuous data streams. To address this, we propose BHAR, a system-level optimization framework for batteryless human activity recognition that learns both the sensor-level data acquisition policy and the downstream activity classification model. Our key insight is that the classification error in batteryless HAR can be decomposed into two components: 1) a passive error governed by the data acquisition policy and 2) an active error dependent on the classifier's performance. Building on this insight, our BHAR framework explicitly minimizes both errors to handle the sparse and asynchronous nature of batteryless sensor data. We validate BHAR on multiple datasets and architectures and show that it closes 39%–59% of the $F1$-score gap between batteryless and battery-powered-HAR compared to baseline approaches. *Code is available at* `https://github.com/SLDGroup/BHAR_codes_artifact`**

***Index Terms*—Activity recognition, batteryless sensors, energy harvesting, Internet of Things, machine learning.**

I. Introduction

BATTERYLESS sensors are transforming smart sensing by providing a sustainable alternative to *battery*-powered sensors [1], [2], [3]. By eliminating batteries, they serve as maintenance-free *data sources* that *do not* require recharging or replacement. While batteries supply continuous power, they require user intervention for replacement or recharging once they die. In contrast, batteryless sensors harvest and store energy from ambient sources, enabling operation without explicit recharging [4]. However, since energy harvesting is an uncertain and discontinuous power source, batteryless sensors only operate when enough energy is stored. As shown in Fig. 1(a), this results in *sparse* and *asynchronous* data streams. This poses difficulties for time-series tasks like human activity recognition (HAR), where most deep learning models expect *synchronized* and *continuous* data across sensors [5], [6], [7].

To enable machine learning (ML) with batteryless sensors, prior works proposed architectures for sparse data [8], algorithms for energy allocation [9], and on-device inference to reduce the energy costs of data transmission [10]. However, these methods consider a *single* sensor and do not address *asynchronous* data across *multiple* sensors. Existing multisensor approaches are limited [11] and use heuristics for data acquisition. In light of these limitations, we cast batteryless HAR as a system-level optimization problem, where data acquisition *and* classification are learned.

Problem Context: Standard HAR classifies human activities (e.g., walk, sit, run) from *synchronized* time series across multiple *battery-powered* sensors [12], with many works [13], [14], [15] focusing on efficient models for edge devices. We call this standard *battery-powered* HAR setting *BP*-HAR. Conversely, we consider *batteryless* HAR with accelerometers that harvest energy from human motion.

Current Limitations: Our setting highlights a critical oversight in prior work: while classifier efficiency is important, *data acquisition* is equally vital when using sensors that rely on energy harvesting. In our setting, low-motion periods create uncertainty in data availability as little or no energy is harvested. This challenge *cannot* be overcome just by designing better activity classification models.

Research Challenge: Thus, ML with batteryless sensors requires optimizing energy use for data acquisition *in addition to* training a deep learning model for sparse, asynchronous data. We outline this optimization problem in Fig. 1(b).

Gap in Existing Work: While prior works in batteryless HAR [16], [17] report high classification accuracy on self-collected data, no works comprehensively compare their approach to a BP counterpart on standard HAR benchmarks.

Received 14 August 2026; accepted 16 August 2026. This work was supported in part by NSF under Grant 2428656 and Grant 2214939, in part by the Office of Naval Research (ONR) under Grant N00014-24-1-2797, and in part by the Intelligent Machine Engineering (iMAGiNE) Consortium at The University of Texas at Austin (UT) Austin. This article was presented at the International Conference on Codesign of Embedded Systems 2026 and appeared as part of the ESWEEK-TCAD Special Issue. This article was recommended by Associate Editor S. Dailey. *(Corresponding author: Geffen Cooper.)*

Geffen Cooper and Radu Marculescu are with the Department of Electrical and Computer Engineering, The University of Texas at Austin, Austin, TX 78712 USA (e-mail: geffen@utexas.edu).

Nathan Tsao, Filippos Fotiadis, and Ufuk Topcu are with the Oden Institute for Computational Engineering and Sciences, The University of Texas at Austin, Austin, TX 78712 USA.

Digital Object Identifier 10.1109/TCAD.2026.3725536

1937-4151 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

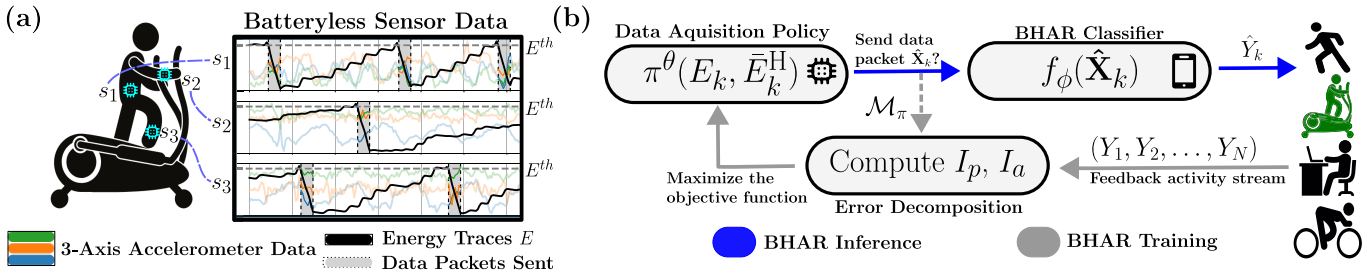


Fig. 1. (a) Batteryless HAR setting. A user wears $b = 3$ batteryless sensors (s_1, s_2, s_3) which harvest energy from motion and output 3-D acceleration (blue, orange, and green signals). The black lines show the energy level of each sensor, which accumulates and then depletes when reaching the threshold E^{Th} for data acquisition (dashed line). The gray boxes show transmitted packets. (b) Batteryless HAR optimization problem. Each sensor has a policy π^{θ} to determine when to send packets $\hat{\mathbf{X}}_k$ to the classifier f_{ϕ}. The classifier runs on a smartphone to *aggregate* data from all sensors and outputs predictions $\hat{Y}_k$ (e.g., run, walk). Using labels $(Y_1, \ldots, Y_N)$ and sent packets $\mathcal{M}_{\pi}$, we split the error into passive (I_p) and active (I_a) components tied to the policy and classifier. Unlike prior works, we optimize data acquisition *and* classification. See Section III for the fully defined notation.

Motivated by this, we systematically analyze the batteryless HAR classification error, shedding light on challenges inherent to ML with batteryless sensors.

Our Contributions: To address the limitations of prior works, we introduce BHAR, a system-level framework for multisensor **B**atteryless **H**uman **A**ctivity **R**ecognition that optimizes *both* the data acquisition policy and the HAR model for classifying sparse, asynchronous data. BHAR learns a data acquisition policy for each sensor, providing the HAR classification model with timely data for accurate activity prediction. Additionally, BHAR enhances existing HAR architectures by introducing *temporal information* at the sensor fusion stage, allowing the classifier to handle asynchronous data without major modifications. We demonstrate that BHAR substantially narrows the performance gap between batteryless and BP systems. Our contributions are as follows.

1) We formalize the *batteryless* HAR task (Section III-A) and propose a unified framework to compare batteryless and BP-HAR on standard benchmarks (Section IV-A).
2) We augment *pretrained* HAR models for asynchronous data by modulating their intermediate feature maps with temporal information from each packet (Section III-B).
3) We decompose the batteryless HAR classification error into *passive* and *active* components and show that they can be directly reduced via the data acquisition policy and HAR classifier, respectively (Section III-C).
4) We test BHAR on multiple HAR datasets and architectures to validate it in multiple settings (Section V).
5) We develop a wearable prototype to validate our experimental setup (Section IV-C).

Overall, we show that optimizing data acquisition and classification significantly closes the gap between batteryless and BP-HAR. The remainder of this article is organized as follows: Section II discusses existing work. Section III defines the batteryless HAR setting and describes our BHAR approach. Section IV describes our experimental framework and hardware prototype. Section V describes our empirical results. Finally, Section VI outlines our main contributions.

II. Related Work and New Contributions

Recent work in batteryless ML has focused on *intermittent computing* for battery-free inference [19], [20], [21] on individual nodes. Our setting is fundamentally different: we view batteryless sensors as *data sources* within a *multisensor* edge AI *system*. We aim to optimize this system for a global objective (HAR classification). We focus this section on settings most relevant to ours and how they differ from our work.

A. Sparse and Irregularly Sampled Time-Series

Few prior works address classification of data *from* batteryless sensors due to scarcity of public data. The main benchmark, the clinical room dataset [22], harvests energy from *antennas* to obtain sparse accelerometer data. Since no energy harvesting traces are available, prior work focuses on *processing* the sparse data, using interpolation techniques [23] or deep learning [8], [24]. In contrast, we consider *data acquisition* as a component to optimize in addition to the classifier. Further, the clinical room dataset uses *one* sensor, while we consider asynchronous data across *multiple* sensors.

Beyond batteryless sensors, other works consider irregularly sampled time-series [25], [26], [27], including asynchronous event cameras [28], [29] and clinical time-series [30], [31]. While these works consider sparse and asynchronous data streams, they focus solely on *classifier optimization*. In contrast, we also optimize *data acquisition*. Further, we unify the batteryless and BP settings and propose a method to augment *pretrained* HAR classifiers for asynchronous data.

B. Energy Allocation Policies

Batteryless sensors must control their energy supply using an energy allocation policy to operate effectively. As such, prior works have explored energy allocation algorithms for energy harvesting devices. In [32], [33], [34], and [35], the authors consider energy-neutral operation (ENO), where the goal is to ensure energy consumption does not exceed harvested energy over a finite time horizon (on the order of hours). Conversely, we focus on energy allocation over short horizons (on the order of seconds). Other works [36], [37] use reinforcement learning to learn an energy management policy for *batteries* recharged via energy harvesting. In contrast, we

work in settings where significantly less energy is stored, preventing prolonged operation when little energy is harvested.

Traditional ENO frameworks are incompatible with our setting due to differences in temporal scale and hardware constraints. ENO algorithms rely on *stateful* tracking over *hours* to manage energy allocation. In batteryless devices, frequent power failures wipe out volatile memory, making it impossible to guarantee the context needed for long-term planning. While *check-pointing* is possible, this falls into a different category of research (intermittent computing [38]). Second, whereas ENO assumes large energy buffers (batteries) capable of retaining energy across large temporal gaps, batteryless devices use tiny capacitors that saturate or deplete very quickly, requiring localized policies for short-term data acquisition.

C. Battery-Powered HAR

Standard HAR, which we synonymously refer to as BP-HAR, has been a focus of active research, addressing challenges such as data heterogeneity across sensors [39], label scarcity [40], and efficient on-device inference [41]. Prior work on efficient HAR [7], [13], [14], [15], [41] focuses on optimizing deep learning architectures, assuming that synchronized data from multiple *BP* sensors is available on a centralized edge device. However, our problem setting highlights a critical gap: while model efficiency and accuracy are important, data acquisition becomes equally crucial when working with extremely constrained sensors.

D. Batteryless HAR Approaches

With advances in ultralow-power hardware, several works have built wearables for batteryless HAR [10], [16], [17]. These works demonstrate the feasibility of batteryless HAR from an implementation perspective. In contrast, we focus on developing better *algorithms* for batteryless sensing systems. In our work, we estimate the amount of energy harvested from public accelerometer data in order to evaluate our approach on standard HAR benchmarks. This enables us to compare the performance of multiple data acquisition policies under identical energy harvesting conditions.

Prior algorithmic works in batteryless HAR [9] consider online energy allocation for the single sensor setting. Conversely, we consider the *multisensor* scenario, and we aim to optimize *both* energy allocation and classification. While [42] considers data acquisition and classification, it uses a heuristic policy. In contrast, we propose a full methodology for optimizing a *parametrized policy* and classifier with comprehensive evaluations on standard HAR benchmarks.

III. Methodology

A. Problem Setting

HAR considers multichannel time series data from b wearable sensors of the form $X = (\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N)$, where $\mathbf{x}_k \in \mathbb{R}^d$ is a vector of samples from d sensor channels at time step k and N is the length of the data stream. We consider three-axis accelerometers with three channels so that $d = 3b$. Thus, sensor $i \in \{1, \ldots, b\}$ supplies data $\mathbf{x}_k^{(i)} \in \mathbb{R}^3$ which contains entries $3(i-1)+1 : 3i$ of $\mathbf{x}_k$. For example, $\mathbf{x}_7^{(2)} \in \mathbb{R}^3$ is data from sensor 2 at time step 7 with entries $(4, 5, 6)$ of $\mathbf{x}_7$.

The data stream X has time-aligned labels $Y = (y_1, y_2, \ldots, y_N)$, where $y_k \in \{1, \ldots C\}$ is the activity performed at time step k and C is the number of activity classes. Since a single time step of data $\mathbf{x}_k$ is insufficient to identify HAR activities, X is buffered into sliding windows $\mathbf{X}_k = (\mathbf{x}_{k-w+1}, \ldots, \mathbf{x}_k) \in \mathbb{R}^{3b \times w}$ of w samples with labels $Y_k = y_k$. Let $(\mathbf{X}_w, \ldots, \mathbf{X}_N)$ be the stream of data windows with corresponding labels $(Y_w, \ldots, Y_N)$, where the first window starts when w samples are buffered. In HAR, the goal is to learn a *classifier* $\hat{Y} = f_\phi(\mathbf{X})$ parametrized by ϕ from a dataset of labeled windows $(\mathbf{X}, Y)$, where $\hat{Y}$ is the prediction of the activity classifier.

1) Multisensor BP-HAR Problem Setting: In battery-powered HAR, we have access to data windows $(\mathbf{X}_w, \ldots, \mathbf{X}_N)$ of synchronized packets across b sensors. Specifically, for each sensor i, we define a window at time k as $\mathbf{X}_k^{(i)} = (\mathbf{x}_{k-w+1}^{(i)}, \ldots, \mathbf{x}_k^{(i)}) \in \mathbb{R}^{3 \times w}$. Thus, $\mathbf{X}_k = \left[\mathbf{X}_k^{(1)}; \ldots; \mathbf{X}_k^{(b)}\right] \in \mathbb{R}^{3b \times w}$ is a stack of *synchronized* windows from each sensor. Practically, a *new* prediction $\hat{Y}_k$ is made from the data $\mathbf{X}_k$ every n_s steps, where n_s is the *stride*. Thus, in multisensor BP-HAR, the prediction $\hat{Y}_k$ follows (1), where k goes from w to N and $\hat{Y}_{w-1}$ is initialized randomly:

$$\hat{Y}_k = \begin{cases} f_\phi(\mathbf{X}_k), & k \bmod n_s = 0 \\ \hat{Y}_{k-1}, & \text{otherwise.} \end{cases} \tag{1}$$

2) Multisensor Batteryless HAR Problem Setting: As shown in Fig. 1(a), a user wears b sensors, which harvest energy from motion. Each sensor transmits data to a central device (a phone), which applies a classifier to the received data. Since each sensor i can only transmit a packet $\mathbf{X}_k^{(i)}$ when sufficient energy is available, the resulting stream of packets for *each* sensor $(\mathbf{X}_{k_1^i}^{(i)}, \mathbf{X}_{k_2^i}^{(i)}, \ldots,)$ is *sparse*, where k_m^i is the time step when the mth data packet is sent from sensor i.

With *multiple* sensors, data becomes asynchronous since two sensors may send their m^{th} packet at different times. Concretely, sensors i and j have sparse and asynchronous streams $(\mathbf{X}_{k_1^i}^{(i)}, \mathbf{X}_{k_2^i}^{(i)}, \ldots,)$ and $(\mathbf{X}_{k_1^j}^{(j)}, \mathbf{X}_{k_2^j}^{(j)}, \ldots)$, respectively, where k_1^i may not equal k_1^j. This is problematic for existing HAR architectures which expect synchronized data across sensors [5], [6], [7]. To address this, we define a *batteryless* HAR pipeline and unify it with BP-HAR, such that existing HAR models can be augmented for sparse, asynchronous data.

B. Policy and Classifier Formulation

1) Data Acquisition Policy: We define data acquisition as sensing *and* transmitting a window $\mathbf{X}_k$ of data. Since a batteryless sensor can only acquire data when sufficient energy is buffered, each sensor i uses a policy $\pi_k^{(i)} \in \{0, 1\}$ at each time step k to decide whether to sense and transmit the *next* packet of data $\mathbf{X}_{k+w}^{(i)} = (\mathbf{x}_{k+1}^{(i)}, \ldots, \mathbf{x}_{k+w}^{(i)})$. Thus, at each time k, the classifier sees the following data from each sensor i:

$$\hat{\mathbf{X}}_k^{(i)} = \begin{cases} \mathbf{X}_k^{(i)}, & \pi_{k-w}^{(i)} = 1 \\ \hat{\mathbf{X}}_{k-1}^{(i)}, & \pi_{k-w}^{(i)} = 0 \end{cases} \tag{2}$$

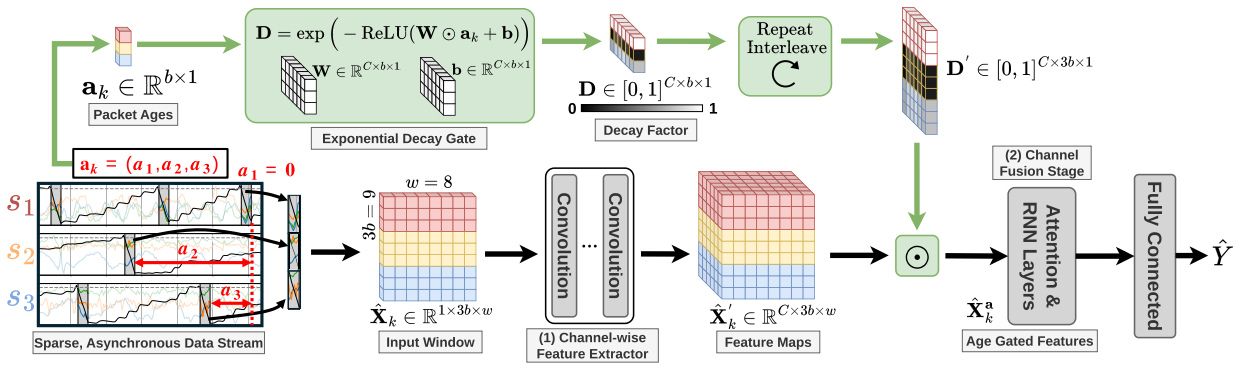


Fig. 2. Multisensor classifier with time context (MSC-T). We show the data flow of asynchronous data for HAR classifiers. The bottom path (black arrows) shows how the input $\hat{\mathbf{X}}_k$ flows in existing HAR architectures, with red, yellow, and blue colors corresponding to channels from sensors s_1, s_2, and s_3, respectively. The top path (green arrows/boxes) shows how we integrate temporal context $\mathbf{a}_k$ to modulate the feature maps. Darker black boxes in the decay factor $\mathbf{D}$ show how stale data will be scaled to zero, while lighter white boxes show how fresh data will be passed through.

where $\hat{\mathbf{X}}_k^{(i)}$ represents the most recent packet received from sensor i at time k, with $\hat{\mathbf{X}}_0^{(i)} = \mathbf{0}$. Specifically, at each step k, the classifier receives a *new* packet $\mathbf{X}_k^{(i)}$ if $\pi_{k-w}^{(i)} = 1$. Otherwise, it sees the last packet sent, i.e., $\hat{\mathbf{X}}_{k-1}^{(i)}$ (stale data), when $\pi_{k-w}^{(i)} = 0$. Optimization of the policy π is often neglected in HAR since data is present at every time step.

2) Batteryless Sensor Model: Here, we define the energy dynamics of the batteryless sensor. The sensor stores energy in a buffer (capacitor) with energy level $E_k \in [0, E^{\max}]$, where k is the time step and $E^{\max} \in \mathbb{R}$ represents the maximum energy the buffer stores. At each step, the sensor harvests E_k^{H} of energy and consumes a fixed amount E^{L}, which accounts for losses such as idle energy usage, the cost to run the policy π, and the cost to sample the voltage of the storage capacitor. Overall, the *energy dynamics* are described as follows:

$$E_{k+1} = E_k + E_k^{\text{H}} - E^{\text{L}} - \pi_k E^{\text{Th}}. \tag{3}$$

Data can only be acquired if the energy in the buffer is higher than a threshold ($E_k > E^{\text{Th}}$). Once this condition is met, the policy decides whether to acquire a packet ($\pi_k = 1$) or remain idle ($\pi_k = 0$) and accumulate energy (assuming $E_k^{\text{H}} > E^{\text{L}}$).

3) BHAR Classifier: The sparse, asynchronous nature of data from the batteryless sensors complicates classification. Packets from each sensor arrive at different times, so we cannot obtain $\hat{Y}_k$ from $\mathbf{X}_k$ as in (1). To address this, we consider three approaches for processing the asynchronous packets.

a) Single Sensor Classifier (SSC): We use a separate model $f_{\phi_i}^{(i)} : \mathbb{R}^{3\times w} \to \{1, \ldots, C\}$ to classify data from each sensor i. The *aggregated single sensor classifier*, $(f_{\phi_1}^{(1)}, \ldots, f_{\phi_b}^{(b)})$, outputs a prediction based on the sensor that sent the most recent packet. We augment (1) for this setting as follows:

$$\hat{Y}_k = \begin{cases} f_{\phi_i}^{(i)}\left(\hat{\mathbf{X}}_k^{(i)}\right), & \exists i \text{ s.t. } \pi_{k-w}^{(i)} = 1 \\ \hat{Y}_{k-1}, & \text{otherwise.} \end{cases} \tag{4}$$

Put simply, we apply classifier $f_{\phi_i}^{(i)}$ if sensor i sends a packet. Otherwise, we keep the previous prediction $\hat{Y}_{k-1}$. For packets received simultaneously, we use $f_{\phi_i}^{(i)}$ with the smallest i.

b) Multisensor Classifier (MSC): We use a multisensor classifier $f_\phi : \mathbb{R}^{3b\times w} \to \{1, \ldots, C\}$ as in standard HAR but redefine the input. We aggregate the most recent packets from each sensor to form an *asynchronous* window $\hat{\mathbf{X}}_k = \left[\hat{\mathbf{X}}_k^{(1)}; \hat{\mathbf{X}}_k^{(2)}; \ldots; \hat{\mathbf{X}}_k^{(b)}\right] \in \mathbb{R}^{3b\times w}$ where $\hat{\mathbf{X}}_k^{(i)}$ is the most recent packet from sensor i as defined in (2). Fig. 2 shows a visualization of this asynchronous window. We augment (1) for this setting as follows:

$$\hat{Y}_k = \begin{cases} f_\phi\left(\hat{\mathbf{X}}_k\right), & \exists i \text{ s.t. } \pi_{k-h}^{(i)} = 1 \\ \hat{Y}_{k-1}, & \text{otherwise.} \end{cases} \tag{5}$$

The prediction updates whenever a new packet is received.

c) Multisensor classifier with time context (MSC-T): Here, we also employ a multisensor classifier $f_\phi : \mathbb{R}^{3b\times w} \to \{1, \ldots, C\}$ with an asynchronous window $\hat{\mathbf{X}}_k \in \mathbb{R}^{3b\times w}$ as input. However, we also include the *age* of each packet $\mathbf{a}_k = (k - k^1, k - k^2, \ldots, k - k^b) \in \mathbb{R}^b$ as an input to condition each packet based on its arrival time relative to the current time step k. We define k^i as the arrival time of *the most recent* packet from sensor i, and $k - k^i$ as the *age of a packet*.

To integrate $\mathbf{a}_k$ into a HAR classifier, we assume a standard two-stage HAR architecture [5], [6], [7]. These have: 1) a channel-wise feature extractor and 2) a channel fusion stage as shown at the bottom of Fig. 2. Stage 1) uses convolution layers to map the input $\hat{\mathbf{X}}_k \in \mathbb{R}^{1\times 3b\times w}$ to features $\hat{\mathbf{X}}'_k \in \mathbb{R}^{C\times 3b\times w}$, where C is the number of feature maps. $\hat{\mathbf{X}}'_k$ is then passed through stage 2) which uses recurrent and attention layers to aggregate information across time and sensor channels.

We use the packet ages to modulate the feature maps $\hat{\mathbf{X}}'_k$. Specifically, we use a learnable exponential decay gate $\mathbf{D} = \exp(-\text{ReLU}(\mathbf{W} \odot \mathbf{a}_k + \mathbf{b}))$ to obtain a **decay factor** $\mathbf{D} \in [0,1]^{C\times b\times 1}$ ($\odot$ is elementwise multiplication, which is applied along the channel dimension). This decay factor (inspired by [43]) ensures that, as a sensor's packets become older, its feature maps get scaled toward zero to reduce the impact of stale data. The parameters $\mathbf{W}$ and $\mathbf{b}$ allow the model to learn distinct feature and sensor-specific decay rates. To match the number of sensor channels ($3b$) in $\mathbf{X}'_k$, we duplicate $\mathbf{D}$ over the sensor dimension to get $\mathbf{D}' \in [0,1]^{C\times 3b\times 1}$. Finally, we apply the decay factor via elementwise multiplication along the time dimension: $\hat{\mathbf{X}}_k^{\mathbf{a}} = \hat{\mathbf{X}}'_k \odot \mathbf{D}'$ where $\hat{\mathbf{X}}_k^{\mathbf{a}} \in \mathbb{R}^{C\times 3b\times w}$ are the age-gated feature maps. Overall, this enables the model to modulate channel-wise features based on the packet arrival times. We augment (1) for this setting as follows:

$$\hat{Y}_k = \begin{cases} f_{\phi,\phi_a}\left(\hat{\mathbf{X}}_k, \mathbf{a}_k\right), & \exists i \text{ s.t. } \pi_{k-w}^{(i)} = 1 \\ \hat{Y}_{k-1}, & \text{otherwise} \end{cases} \tag{6}$$

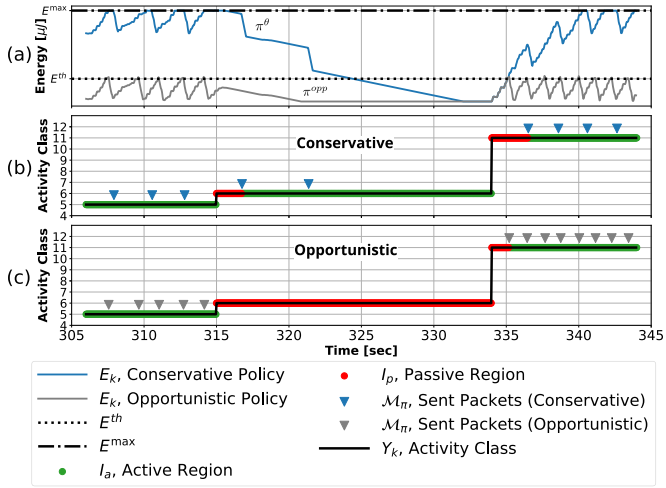


Fig. 3. (a) Energy level E_k for two policies. The dotted line is the threshold E^{Th} needed to send a packet. The dashed line is the max energy $E^{\max}$. The opportunistic policy (gray), π^{opp}, transmits when $E_k > E^{\text{Th}}$. The conservative policy (blue), π^{θ}, saves an energy surplus. (b) and (c) Active I_a (green) and passive I_p (red) regions of π^{θ} and π^{opp}, respectively. The packet *transmission times*, $\mathcal{M}$, are shown as triangles. During a low motion activity (315–334 s), we have $E_k^{\text{H}} < E^{\text{L}}$ so E_k decays. Thus, π^{opp} cannot send packets while π^{θ} uses its surplus, resulting in a large and small I_p, respectively.

where f_{ϕ,ϕ_a} is a multisensor classifier f_ϕ that has additional parameters ϕ_a to process the input $\mathbf{a}_k$. We visualize this data flow in Fig. 2. Furthermore, Section V shows ablations *without* time context to validate the benefit of using time as an input.

The difference between each classifier is the aggregation strategy used to handle asynchronous packets. The single-sensor classifier (SSC) assigns *all weight* to the single, most recently active sensor, treating data from other sensors as stale. The multi-sensor classifier (MSC) assigns *equal weight* to all sensors, treating older and newer packets equally. Our Multi-Sensor Classifier with Time Context (MSC-T) *dynamically weights* packets based on their arrival times.

C. Active and Passive Error Decomposition

In this section, we formulate and analyze the problem of optimizing HAR performance for batteryless sensors. We decompose the HAR classification error into two components, which we reduce by optimizing the data acquisition policy and HAR classifier, respectively. We first introduce key definitions.

1) Transmission Times and Activity Transitions: Given a data acquisition policy $\pi^{(i)}$ with $i \in \{1, \ldots, b\}$ as the sensor index, the set of *transmission times* for sensor i up to time N, $\mathcal{M}_\pi^{(i)}(N)$, denotes when a packet is sent from sensor i

$$\mathcal{M}_\pi^{(i)}(N) = \left\{ k \in \{1, \ldots, N\} \mid \pi_k^{(i)} = 1 \right\}. \tag{7}$$

The set of *all transmission times* from all sensors up to time N, $\mathcal{M}(N)$, is defined as the union of all $\mathcal{M}_\pi^{(i)}(N)$

$$\mathcal{M}(N) = \bigcup_{i=1}^{b} \mathcal{M}_\pi^{(i)}(N). \tag{8}$$

Given activity labels $Y = (Y_1, \ldots, Y_N)$, the set of times when the activity *transitions*, $\mathcal{A}_Y$, is

$$\mathcal{A}_Y = \{k \in \{2, \ldots, N\} \mid Y_k \neq Y_{k-1}\}. \tag{9}$$

2) Active and Passive Regions: Next, we observe that a HAR data stream can be split into *passive* and *active* regions (I_p and I_a). For brevity, we denote $I_p(Y)$ as I_p and likewise for $I_a(Y)$, though both regions depend on the labels Y and policy π. The **passive region** (I_p) is the set of time steps following an activity transition when the classifier has not yet received packets reflecting the transition. This can occur when no sensor has enough energy to transmit, resulting in outdated predictions. Conversely, the **active region** (I_a) is all other time steps when at least one sensor has transmitted data during the activity. Formally, we define these as follows:

$$I_p = \bigcup_{j=1}^{|\mathcal{A}_Y|} \left\{ \mathcal{A}_Y^j, \ldots, \min\left(k \in \mathcal{M}(N) \mid k > \mathcal{A}_Y^j \right) \right\} \tag{10}$$

$$I_a = \{1, \ldots, N\} \setminus I_p \tag{11}$$

where $\mathcal{A}_Y^j$ are the elements of $\mathcal{A}_Y$ with index $j \in \{1, \ldots, |\mathcal{A}_Y|\}$. In (10), $\min(k \in \mathcal{M}(N) \mid k > \mathcal{A}_Y^j)$ represents the first time data was transmitted to the classifier after activity transition $\mathcal{A}_Y^j$. To clarify how these regions are defined, we visualize I_p (red) and I_a (green) in Fig. 3.

3) Error Decomposition: We now show how to decompose the BHAR error into *passive* and *active* errors, which are errors that arise in the passive and active regions, respectively.

1) The error of the HAR classifier can be expressed as the probability of a misclassification for a time step k

$$\epsilon = P\left(\hat{Y}_k \neq Y_k\right).$$

2) By the law of total probability, we decompose the probability of a misclassification event as follows:

$$\begin{aligned} \epsilon &= P\left(\hat{Y}_k \neq Y_k\right) \\ &= P\left(\hat{Y}_k \neq Y_k \,|\, k \in I_a\right) P\left(k \in I_a\right) \\ &\quad + P\left(\hat{Y}_k \neq Y_k \,|\, k \in I_p\right) P\left(k \in I_p\right) \end{aligned}$$

where landing in the active ($k \in I_a$) and passive ($k \in I_p$) regions are disjoint events that partition an activity sequence.

3) Now, for a classifier f, the probabilities of misclassifying an activity (with label Y_k at time k) within I_a and I_p are

$$\begin{aligned} \epsilon_a &= P\left(\hat{Y}_k \neq Y_k \,|\, k \in I_a\right) \\ \epsilon_p &= P\left(\hat{Y}_k \neq Y_k \,|\, k \in I_p\right). \end{aligned}$$

When substituting these *conditional error rates* ϵ_a and ϵ_p into the expression from 2), we get

$$\epsilon = (\epsilon_a)\, P\left(k \in I_a\right) + \left(\epsilon_p\right) P\left(k \in I_p\right). \tag{12}$$

We expect ϵ_p to be high because, to *correctly* classify an activity in I_p, the classifier must first misclassify an activity in I_a and then transition to the activity it incorrectly predicted. We want ϵ_a to be low because it is the classifier's error rate when it has received updated packets from the policy.

4) Now, for a policy π, the probability of landing in the passive and active regions can be estimated empirically

from the training data as shown below. We call these probabilities the *normalized* active and passive regions

$$P(k \in I_a) = \frac{|I_a|}{|I_a| + |I_p|}, \quad P\left(k \in I_p\right) = \frac{|I_p|}{|I_a| + |I_p|}.$$

5) Substituting the result of 4) into (12), the classification error ϵ of f is *decomposed* as follows:

$$\epsilon = \underbrace{(\epsilon_a) \frac{|I_a|}{|I_a| + |I_p|}}_{\text{active error}} + \underbrace{\left(\epsilon_p\right) \frac{|I_p|}{|I_a| + |I_p|}}_{\text{passive error}}. \tag{13}$$

We now describe how the policy and classifier contribute to the passive and active error, respectively. As discussed in 3), the classifier generally achieves lower error during the active region ($\epsilon_a < \epsilon_p$). Under this assumption, the *passive error* can be reduced by minimizing the probability of landing in the passive region, equivalently, by maximizing the active region. This effectively increases the weight of ϵ_a (the smaller component) in the overall error. To achieve this, we define the following objective for the *policy* to maximize:

$$R_\pi(Y) = \frac{\left|I_a(Y)\right|}{\left|I_a(Y)\right| + \left|I_p(Y)\right|}. \tag{14}$$

Intuitively, the policy is optimized to maximize the active region, ensuring timely predictions. To reduce *active error*, we optimize the classifier to minimize misclassifications within the active region (i.e., minimize ϵ_a). We now explain how we optimize the policy and classifier in Sections III-D and III-E.

D. Policy Optimization: Reducing Passive Error

In batteryless sensing, an *opportunistic* policy (π^{opp}) is common [10], [16] where sensors operate *whenever* E_k (energy stored at time k) surpasses a required threshold E^{Th}

$$\pi_k^{opp} = \begin{cases} 1, & E_k > E^{\text{Th}}, \ \left(\text{energy above threshold}\right) \\ 0, & \text{otherwise}. \end{cases} \tag{15}$$

Intuitively, π^{opp} is greedy as it acquires data whenever sufficient energy is available and does not save any energy. Thus, when the user transitions to a dormant activity like sitting, the policy will not have enough energy to send a packet to the classifier, thus incurring a large passive error.

To reduce passive error compared to π^{opp}, we introduce a *conservative* policy π^θ parameterized by $\theta = (\theta_E, \theta_P)$. Intuitively, π_k^θ sends a packet if sufficient energy is available *and* a minimum time interval τ^θ has elapsed since the last packet was sent

$$\pi_k^\theta = \begin{cases} 1, & E_k > E^{\text{Th}} \wedge k - \max \mathcal{M}^{(i)}(k) \geq \tau^\theta \\ 0, & \text{otherwise} \end{cases} \tag{16}$$

where $k - \max \mathcal{M}^{(i)}(k)$ is the time since the last packet was sent. The dynamic threshold τ^θ is defined as a function of the current energy surplus $\tilde{E}_k$ and the harvesting rate $\bar{E}_k^{\text{H}}$

$$\tau^\theta = \left(1 - \tilde{E}_k\right) \theta_P \bar{E}_k^{\text{H}} + \theta_E. \tag{17}$$

The energy surplus $\tilde{E}_k$ is the energy level E_k rescaled

$$\tilde{E}_k = \frac{E_k - E^{\text{Th}}}{E^{\max} - E^{\text{Th}}}. \tag{18}$$

Consequently, $\tilde{E}_k = 0$ when the stored energy lies at the minimum operating threshold ($E_k = E^{\text{Th}}$), and $\tilde{E}_k = 1$ when the energy storage is fully saturated ($E_k = E^{\max}$). In other words, $\tilde{E}_k$ represents the energy surplus. Note that when $E_k < E^{\text{Th}}$, the threshold τ^θ is not considered since (16) uses an $\wedge$ (and) operation.

The value $\bar{E}_k^{\text{H}}$ is defined as the amount of energy harvested in the last second. This provides a simple way to estimate the energy harvesting rate (μJ/s) as the time unit in the denominator is one. This value is also rescaled by dividing by the largest value in the training data.

Overall, the conservative policy π_k^θ sends a packet if sufficient energy is available *and* sufficient time has passed since the last packet was sent. Since τ^θ is a function of $\tilde{E}_k$ and $\bar{E}_k^{\text{H}}$, the policy can be more or less conservative based on the energy stored and harvested *over time*. The policy can also learn to be opportunistic as $\pi^\theta = \pi^{opp}$ when $\theta = (0, 0)$. The parameterization of the threshold τ^θ allows the policy to conserve energy when the energy surplus is low and the harvesting rate is high, but then transition to a constant delay θ_E when the energy surplus is maximized at $\tilde{E}_k = 1$.

We aim to train π^θ to maximize R_π. However, the objective R_π is not differentiable with respect to the policy parameters θ, so we optimize π^θ using the pattern search method [44], a derivative-free optimization method [45]. Given parameters θ and objective function R_π, the pattern search method (in Section IV-E) searches for a local extreme point by evaluating points along certain directions. Then, it either steps toward the direction with a higher value, if such a direction exists, or reduces the step length. The pattern search method has exponential runtime in the number of parameters ($2 \cdot b$ in our case). Thus, we parallelize the pattern search to optimize all b conservative policies simultaneously. The parallelized search algorithm updates the parameters of one sensor each iteration, reducing the runtime to be linear in the number of sensors b.

E. Classifier Optimization: Reducing Active Error

Next, we aim to reduce *active error* by minimizing misclassifications within the active region (minimize ϵ_a). We train the multisensor classifier with time context [MSC-T, (6)] to minimize the cross entropy (CE) loss in the active region on a labeled asynchronous dataset $\{((\hat{\mathbf{X}}_j, \mathbf{a}_j), Y_j)\}_{j=1}^n$. We construct this dataset by applying a policy to standard HAR data to obtain a sparse, asynchronous data stream. By definition of the active region, every training input $\hat{\mathbf{X}}$ is in I_a. Thus, we minimize misclassifications within the active region via the following objective for the MSC-T:

$$\min_{\phi, \phi_a} \sum_{j=1}^{n} \mathcal{L}_{CE}\left(f_{\phi, \phi_a}\left(\hat{\mathbf{X}}_j, \mathbf{a}_j\right), Y_j\right). \tag{19}$$

IV. Experimental Setup

A. Unifying Batteryless and BP-HAR

We unify the batteryless and BP-HAR settings by describing them as a composition of: 1) a data acquisition policy and 2) a classification model. To evaluate our BHAR framework within this design space, we define two baselines that act as upper and lower bounds for batteryless HAR.

1) BP-HAR: In this setting, the policy has no energy constraints and the classifier is multisensor. It is applied to synchronized data windows as in (1) with the stride set to half the window length, $n_{\text{stride}} = \lfloor w/2 \rfloor$. We treat this baseline as the *upper bound*.

2) Opportunistic Policy With Single Sensor Classifier (Opp-SSC): As a *lower bound*, we use the opportunistic policy [see (15)] with the single sensor classifier [see (4)]. We consider this the lower bound because the policy is not optimized and the classifier does not consider correlations *across* sensors. *Note*: all sensors are available in this baseline (and all other baselines). The classifier only considers data from the most recently active sensor, as this is the simplest aggregation scheme for asynchronous packets.

3) Conservative Policy With MSC-T (Cons-MSC-T): This approach optimizes the parameters of a conservative policy [see (16)] to maximize the active region [see (11)] and fine-tunes a multisensor classifier with time context (MSC-T) [see (6)]. We validate this approach empirically, showing how it reduces active and passive errors compared to (Opp-SSC).

B. Data Preparation

We convert standard HAR benchmarks into batteryless HAR data by: 1) estimating harvested energy from raw accelerometer traces and 2) applying our energy dynamics model [see (3)] with a given policy to generate sparse accelerometer data. Note, this process is validated via our hardware prototype (Section IV-C). We apply this conversion process and evaluate on four multisensor HAR benchmarks.

1) *DSADS [46]:* The Daily Sports and Activities Dataset contains 19 activities from 8 subjects collected using 5 inertial sensors on the torso, both arms, and both legs.
2) *RWHAR [47]:* The Real World HAR dataset has 8 activities performed by 15 subjects collected from 7 inertial sensors on the head, chest, forearm, shin, thigh, upper arm, and waist.
3) *PAMAP2 [48]:* The Physical Activity Monitoring dataset has 18 activities from 9 subjects collected using 3 inertial sensors on the chest, hand, and ankle.
4) *OPPORTUNITY [49]:* The Opportunity dataset has several sensor types and activity *levels*. We use the *locomotion* activity level, which has 5 activities from 4 subjects. We consider 7 of the on-body inertial sensors from the back, arms, and shoes.

For *DSADS*, *RWHAR*, and *PAMAP2*, data were collected per activity, so we create activity sequences by randomly sampling and concatenating 10–30 s segments (we consider other activity durations in Section V-C). Conversely, subjects in *OPPORTUNITY* sequentially performed activities in a natural manner, so we use the sequences directly.

In Table I, we summarize the datasets used in our experiments. For each dataset, we only use the accelerometer sensors (no gyroscope) due to the constraints of BLE (Bluetooth low energy)-based batteryless *beacons*. Accelerometers are much lower power than gyroscopes [50], making them more feasible for batteryless devices (see Section IV-C). Further, BLE beacons can only transmit 31-byte packets. After accounting for length and type header bytes, the payload accommodates a maximum of nine samples of accelerometer data, where each sample is 3 bytes (one byte per X, Y, Z channel). In our experiments, we use a window size of 8 samples.

TABLE I
SUMMARY OF DATASETS

Name	# Activities	# Subjects	Sampling Rate	# Sensors	Sensors Used
DSADS	19	8	25 Hz	5	Accelerometer
RWHAR	8	13	50 Hz	7	Accelerometer
PAMAP2	12	8	100 Hz	3	Accelerometer
OPPORTUNITY	5	4	30 Hz	7	Accelerometer

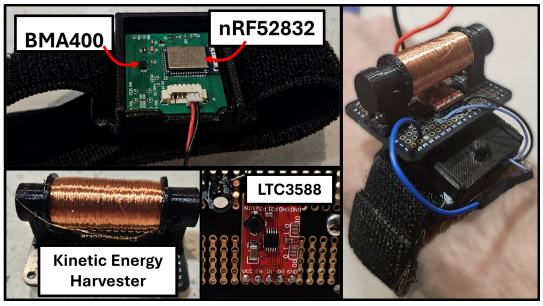


Fig. 4. Hardware prototype. The PCB with a BMA400 accelerometer and nRF52832 BLE MCU in a wearable case (top left). The KEH: a wound coil with a magnetic core moving inside (bottom left). The LTC3588 PMIC: rectifies the AC voltage from the KEH to charge a 10 μF capacitor, and regulates the voltage to 1.8 V to power the PCB (bottom middle). The integrated prototype is worn on the wrist (right).

We resample the raw accelerometer data from each dataset to 25 Hz. For the RWHAR dataset, we do not include subjects 2, 3, 6, and 8 as these subjects either do not have data for certain sensors or have corrupted data. For the PAMAP2 dataset, we do not include subject 9 because they complete very few of the activities in the dataset. Also, PAMAP2 has activities that very few subjects perform. Thus, similar to prior works [7], we remove these activities.

C. Batteryless HAR Data

1) Energy Harvesting: We estimate the kinetic energy harvested from raw accelerometer traces via a widely used mass-spring-damper model [51], [52]. Following [53], we convert the accelerometer signal magnitude into the displacement of the harvester's proof mass, from which power and energy are derived. We assume an energy conversion efficiency of 0.5. We apply this procedure to obtain E_k^{H} for **each** individual accelerometer. These energy harvesting traces are then applied to the energy dynamics from (3).

We validate this approach by implementing a kinetic energy harvester (KEH) and wearable prototype (Fig. 4). The KEH powers the prototype, which acquires a packet of accelerometer data once E^{Th} of energy is harvested. We measure the *interarrival* time between successive packets that are *received* by an nRF52 device [54]. In Fig. 5, we compare the distribution of interarrival times from our prototype (orange bars) and the harvesting model (blue bars) when walking with the sensor on the leg and arm. The similarity between the distributions and their means indicates that the harvesting model provides a practical estimate of harvested energy. We validate further via a sensitivity analysis in Section V-C.

2) Energy Consumption: We use the PPK2 [55] to profile our prototype, which uses an nRF52832 BLE microcontroller

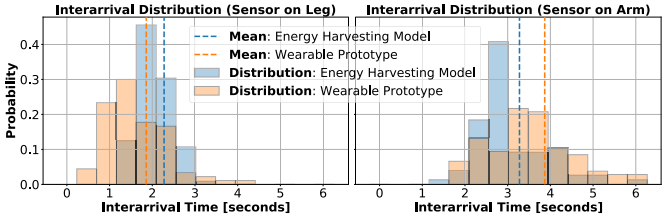


Fig. 5. Interarrival time distribution of energy harvesting model (blue) and prototype (orange) while walking. The similarity of the distributions reflects that the harvesting model provides a practical estimate of harvested energy.

TABLE II
BATTERYLESS SENSOR MODEL PARAMETERS

Description	Name	Value
(Sleep + Policy)	E^{L}	$6.6\mu W \cdot \frac{1}{25} sec = 0.264\mu J$
(Sensing + BLE Transmission)	E^{Th}	$(22+28)\mu J = 50\mu J$
Maximum Energy Storage	$E^{\max}$	$200\mu J$

(MCU) [56], a BMA400 accelerometer [57], and an LTC3588 power management IC (PMIC) [58]. In our setup, the KEH charges a 10 μF capacitor through the LTC3588, which regulates the voltage to 1.8 V and disconnects power when the capacitor voltage drops to 2.87 V.

Policy evaluation consists of sampling the capacitor voltage and executing policy logic. With a time step k of 40 ms (accelerometer sampling period), we evaluate the policy every five time steps (200 ms). The system sleeps otherwise. Overall, the average power consumption during sleep and policy execution is 6.6 μW. We find that the overhead of the conservative policy is negligible compared to the opportunistic policy, so we use the same value for E^{L} in our experiments. Finally, the energy used to read eight accelerometer samples (22 μJ) and transmit over BLE (28μJ) results in a data acquisition cost of 50 μJ. All parameters are shown in Table II.

D. Training Procedures

1) Pre-Training: We pretrain three standard multisensor HAR models: *DeepConvLstm* [6], *Attend and Discriminate* [5], and *TinyHAR* [7], abbreviated as `attend`, `convlstm`, and `tinyhar` respectively. For each dataset and architecture, we pretrain a single-sensor model on each body part and a multisensor model on all body parts. We use leave-one-subject-out-cross-validation (LOSOCV), which trains on $S-1$ subjects and tests on the remaining subject, where S is the number of subjects. We repeat LOSOCV across three random seeds. We use 10% of the training set for validation to tune hyperparameters.

We train for 25 epochs with the Adam optimizer (PyTorch default) using a batch size of 256 and a learning rate of 0.0001 with a cosine annealing scheduler. The window size is 25 samples for BP-HAR and eight samples otherwise. The stride is set to half the window size.

In the original papers [5], [6], [7], the authors use large windows (1–5 s), use gyroscope data, and evaluate performance at the *window* level. Thus, they report higher $F1$-scores than our BP-HAR results. In contrast, our BP-HAR setup uses a 1 s window and evaluates performance at the *sample* level over the data stream. We avoid large windows as they accumulate passive errors during activity transitions. For all other settings, we use a batteryless sensor, which is limited to an eight-sample window (320 ms). To augment the architectures for this small window size, we adjust the initial convolution layers by using a kernel size of 3 with padding, such that the window stays the same length after each convolution layer.

2) Post-Training: After pre-training, we apply the trained classifiers to each setting outlined in Section IV-A as follows.

1) *Obtain Data Acquisition Policies* $\pi^{(i)}$: If the desired policy is conservative, we train it as outlined in Section III-D (more details in Section IV-E). If the policy is unconstrained (the BP-HAR setting) or opportunistic, no training is needed.
2) *Create Sparse, Asynchronous Datasets:* We apply a policy $\pi^{(i)}$, based on the energy dynamics in (3), to each HAR dataset to obtain the batteryless HAR data.
3) *Obtain a Batteryless HAR Classifier:* If the classifier is multisensor, we fine-tune a pretrained classifier using (19). If it is a single-sensor classifier, no fine-tuning is needed.
4) *Evaluate:* We apply the *trained* policy and classifier to the test subject's data to get predictions $\hat{Y}$ and get the macro-averaged $F1$-score, accuracy, and active and passive errors. For post-training, we use LOSOCV.

We now elaborate on the fine-tuning procedure of post-training. Given that the single-sensor models can be applied directly to the sparse, asynchronous data, fine-tuning is only required for multisensor models (with/without time context). After creating a dataset of asynchronous windows, we apply a similar training procedure as pre-training. We train the multisensor models using similar hyperparameters as pre-training but with a batch size of 32 and 10 epochs.

We use a *weighted random sampler* to form batches during post-training to address class imbalance introduced by the data acquisition policy, which undersamples low-motion activities like sitting. The goal of the sampler is to form batches that are approximately class-balanced. We use weighted sampling for DSADS, RWHAR, and OPPORTUNITY, and uniform sampling for PAMAP2.

E. Policy Training Details

In Algorithm 1, we describe the pattern search method [44], the derivative-free optimization algorithm [45] we used to optimize the conservative policy π^{θ} [see (16)]. Here, $R_{\pi}(Y;\theta)$ is the objective function to maximize [see (14)], which depends on the activity labels Y and policy parameters θ. $\mathcal{D}_k$ is the set of valid perturbation directions for the parameter θ_k at iteration k, where a direction p_k is in $\mathcal{D}_k$ if the perturbation keeps the updated parameter $\theta_{k+1} = \theta_k + \gamma_k p_k$ within the parameter bounds $\mathcal{B}$. Thus, the direction set is defined as $\mathcal{D}_k = \{p_k \,|\, \theta_{k+1} = \theta_k + \gamma_k p_k \in \mathcal{B}\}$.

On each iteration k, the pattern search method (Algorithm 1) evaluates the function at $\theta_k + \gamma_k p_k$ for all $p_k \in \mathcal{D}_k$ in a batch of size B. If there exists a direction p_k that has a better average value than the initial point θ_k, then the new point θ_{k+1} is stepped along p_k and the step length γ_k is scaled by $\Phi_k \geq 1$.

Algorithm 1 PATTERNSEARCH $(\theta; \gamma, \alpha, \Phi, N)$: Derivative-Free Method to Find $\theta^* = \operatorname{argmax} R_{\pi^\theta}$

Parameters: $\alpha_k \leq 1$ (contract), $\Phi_k \geq 1$ (expand)
Initialize: θ_0 (policy), γ_0 (step length)
for $k = 0, \ldots, N-1$ **do**
  if $\sum_{j=1}^{B} R_\pi(Y_j; \theta_k + \gamma_k p_k) > \sum_{j=1}^{b} R_\pi(Y_j; \theta_k)$ for some direction $p_k \in \mathcal{D}_k$ **then**
    $\theta_{k+1} \leftarrow \theta_k + \gamma_k p_k$
    $\gamma_{k+1} \leftarrow \Phi_k \gamma_k$ // *Expansion step*
  else
    $\theta_{k+1} \leftarrow \theta_k$
    $\gamma_{k+1} \leftarrow \alpha_k \gamma_k$ // *Contraction step*
  end if
end for
return $\left(\theta_{N-1}, \frac{1}{B}\sum_{j=1}^{B} R_\pi(Y_j; \theta_{N-1})\right)$ $\{\theta^*$, avg reward$\}$

Algorithm 2 PARALLELPATTERNSEARCH$(\theta; \gamma, \alpha, \Phi, N)$: Parallel Optimization of All Policie

Parameters: $\alpha^{(i)} \leq 1, \Phi_k^{(i)} \geq 1$ for sensors $i \in \{1, \ldots, b\}$
Initialize: $\theta_0^{(i)}, \gamma_0^{(i)}$ for sensors $i \in \{1, \ldots, b\}$
for $k = 0, \ldots, N-1$ **do**
  // *Run 1 PatternSearch epoch for all sensors in parallel*
  $\{(\tilde{\theta}^{(i)}, \tilde{R}^{(i)})\}_{i=1}^{b} \leftarrow \text{PSearch}(\theta_k^{(i)}; \gamma^{(i)}, \alpha^{(i)}, \Phi_k^{(i)}, 1)$
  // *Only update sensor j with the best reward*
  $(\theta_{k+1}^{(j)}, j) \leftarrow \max_i \{(\tilde{\theta}^{(i)}, \tilde{R}^{(i)})\}_{i=1}^{b}$
  // *Keep parameters the same for other sensors*
  $\theta_{k+1}^{(i)} \leftarrow \theta_k^{(i)}$ for all $i \neq j$
end for
return $\{\theta_N^{(i)}\}_{i=1}^{b}$// *Return all optimized parameters*

Otherwise, γ_k is scaled by $\alpha_k \leq 1$. We set $\alpha = 0.5$, $\phi_k = 1.5$, $\theta_0 = (0, 0)$, $\gamma_0 = 100$, and

$$\mathcal{D}_k = \left\{ \frac{(d_1, d_2)}{\|(d_1, d_2)\|_2^2} \in \mathbb{R}^2 \;\middle|\; (d_1, d_2) \in \{-1, 0, 1\}^2 \right. \\ \left. \wedge\, \forall p_k \in \mathcal{D}_k \colon \theta_k + \gamma_k p_k \in \mathcal{B} \right\}$$

$$\mathcal{B} = \left\{ (b_1, b_2) \in \mathbb{R}^2 \mid (b_1, b_2) \in (-1000, 1000)^2 \right\}.$$

However, pattern search (Algorithm 1) has exponential run-time since the direction set $\mathcal{D}_k$ grows exponentially with the number of parameters. Thus, we cannot optimize all b policies simultaneously using Algorithm 1, as there will be $2b$ parameters (θ_E and θ_P for each sensor).

We circumvent the exponential run-time by parallelizing the pattern search method. During iteration k of the parallelized method (Algorithm 2), a normal pattern search is run for one epoch in parallel across all batteryless sensors. Then, instead of updating the parameters of all the sensors $\theta^{(i)}$ simultaneously, we *only* update the parameters of the sensor j that achieved the greatest improvement with respect to the objective function. Thus, the resulting parallelized algorithm (Algorithm 2) has linear runtime with respect to the number of sensors b.

V. EMPIRICAL RESULTS AND ABLATIONS

A. Validation Against Performance Bounds

To empirically validate our approach, we plot the average (over seeds and subjects) accuracy, macro $F1$-score, and active/passive error breakdowns for all datasets and architectures in Fig. 6. For each plot, the black and gray bars correspond to the upper (*BP-HAR*) and lower (*Opp-SSC*) bound baselines, respectively, while the blue bars show the result of optimizing *both* the policy and classifier *conservative policy with MSC-T (Cons-MSC-T)*.

Fig. 6 explicitly maps how our *Cons-MSC-T* method improves over the lower bound (*Opp-SSC*), and measures how close we come to the upper bound (*BP-HAR*) by splitting each error bar into its active and passive components.

1) *Passive Error (Red Bars) → Policy's Impact:* By looking at how the red bars shrink from *Opp-SSC* to *Cons-MSC-T*, we see the amount of error reduced due to our *conservative policy*. Specifically, there is a 4.25% reduction in *passive* error on average across all settings.
2) *Active Error (Green Bars) → Classifier's Impact:* By looking at how the green bars shrink from *Opp-SSC* to *Cons-MSC-T*, we see the amount of error reduced due to our *multisensor classifier with time context*. Specifically, there is a 6.6% reduction in *active* error on average across all settings.

As shown in Sections III-E and III-D, the *active* error is reduced by optimizing the classifier to minimize the active error rate ϵ_a, and the *passive* error is reduced by optimizing the policy to minimize the passive region. Thus, reduction in active and passive errors reflects improvements in the classifier and the policy, respectively. This validates that minimizing the passive region and mispredictions in the active region is effective in reducing the classification error.

The $F1$-score *gap* between the lower and upper bound is 20%–32% across all settings, reflecting the difficulty of batteryless HAR. This stems from the low *passive* error of *BP-HAR*, which acquires data at most time steps. Overall, we see that *Cons-MSC-T* improves over the *Opp-SSC* $F1$-score by 10%–14% (absolute points), closing 39%–59% of the gap.

Looking closer at the active/passive error breakdown in Fig. 6, we see that across all results, the *active* error (green) *gap* between BHAR and BP-HAR is 0%–7% and is closed almost completely for the RWHAR and OPPORTUNITY benchmarks. Conversely, the *passive* error (red) *gap* between BHAR and BP-HAR is 8%–16%. The larger *passive* error gap indicates that the performance bottleneck is rooted in the timeliness of predictions rather than classifier performance alone. Put simply, improving the classifier yields diminishing returns if the underlying policy fails to provide timely data; the model's performance is fundamentally capped by the availability of the data. This emphasizes the importance of using a *system-level* approach that optimizes data acquisition at the *sensor-level* to minimize *passive* error.

B. Detailed Error Metrics and Ablation

To validate each component in our framework, Figs. 7–10 show detailed metrics for *every* policy-classifier combination across all datasets (we limit results to the TinyHAR

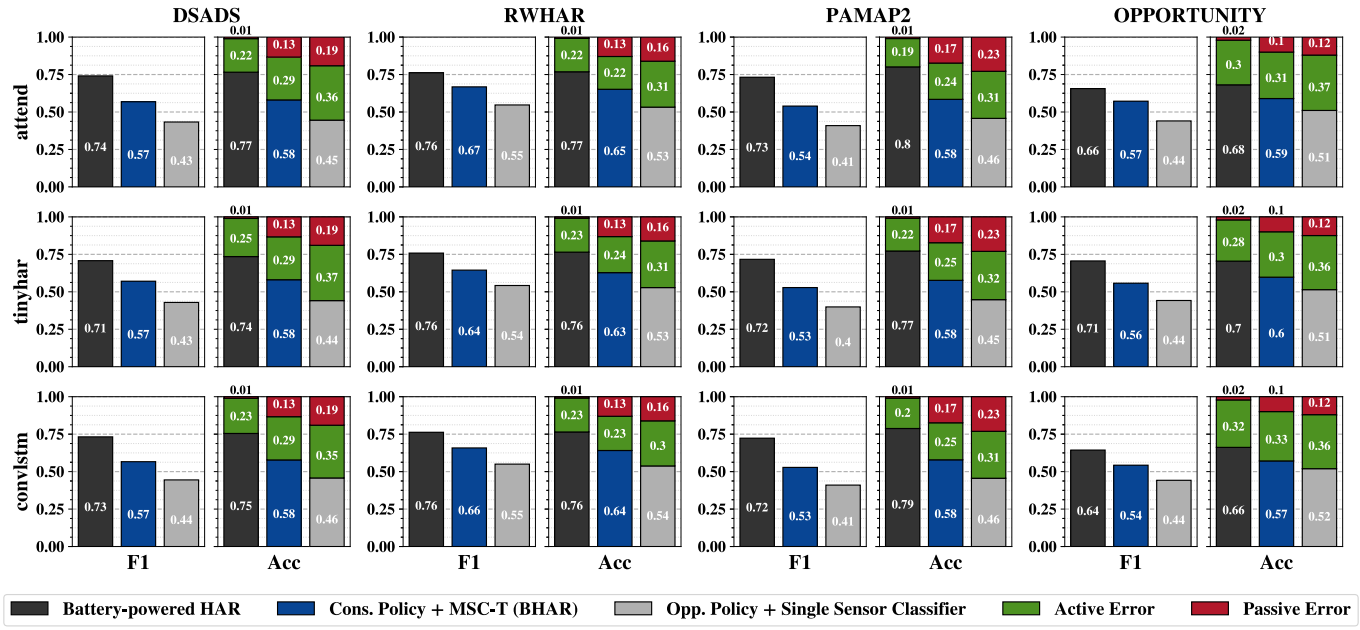


Fig. 6. $F1$-score and accuracy breakdown. Each row (e.g., attend, etc.) is an architecture, while each *pair* of columns shows the $F1$-score ($F1$) and accuracy (Acc) for each dataset (e.g., DSADS, etc.). The black and gray bars show the upper and lower bound baselines, respectively, while the blue bars show Cons-MSC-T. The error (green + red bars) and accuracy add to 1 as shown by the stacked bars reaching 1 in the Acc columns. The reduction in active (green) and passive (red) errors reflects improvements in the classifier and the policy, respectively. Opp. and Cons. stand for opportunistic and conservative, respectively.

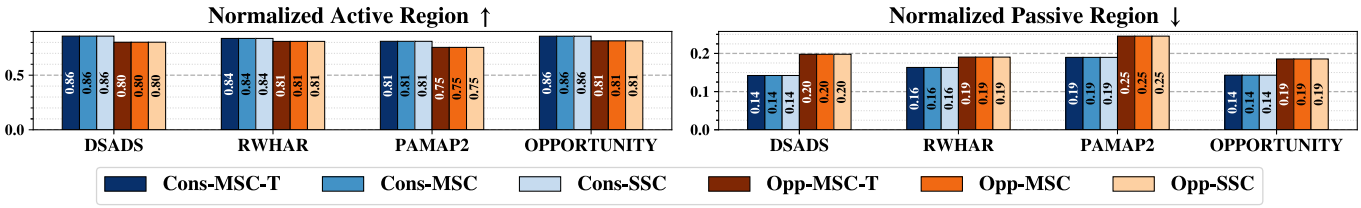


Fig. 7. Normalized active (left) and passive (right) *regions* for all policy-classifier combinations. The normalized *active* region is *larger* (normalized *passive* region is *lower*) for the conservative (blue) relative to the opportunistic approaches (orange).

architecture). We evaluate six configurations across *two policies*, opportunistic (Opp) and conservative (Cons), and *three classifiers*, SSC, MSC, and multisensor with time context (MSC-T). In each figure, the title shows the metric evaluated (on the y-axis), where ↑ means higher is better, and ↓ means lower is better. In each plot, a group of six bars shows the results for a dataset. The three *blue* bars are methods that use a **conservative** policy while the three *orange* bars are methods that use an **opportunistic** policy. In each policy (color) group, the bars are ordered from light to dark to show how results differ across classifiers from single sensor, to multisensor, to multisensor with time context.

1) Normalized Active/Passive Regions: In Fig. 7 we plot the active/passive region breakdown. These are empirical estimates of $P(k \in I_a)$ and $P(k \in I_p)$ and serve as direct evaluations of the data acquisition policy. For each dataset, we see that the normalized active region is larger (and the normalized passive region is lower) for the conservative (blue) approaches relative to the opportunistic approaches (orange). Specifically, we see that the conservative policy achieves a 5% lower passive region on average compared to the opportunistic policy. This reflects how the objective in (14) was successfully optimized. Since these metrics are independent of the classifier, they are the same across a policy group.

2) Active/Passive Error Rates: In Fig. 8, we plot the error rates within the active and passive regions which are empirical estimates of $P(\hat{Y}_k \neq Y_k | k \in I_a)$ and $P(\hat{Y}_k \neq Y_k | k \in I_p)$. The active error rate ϵ_a directly evaluates the classifier, as it represents the error rate in the active region. For each policy, the multisensor classifier with time context (MSC-T) achieves a lower ϵ_a than the MSC, which achieves a lower ϵ_a than the SSC. Specifically, MSC approaches achieve a 2.7% lower ϵ_a compared to SSC approaches, while MSC-T approaches achieve an 8.3% lower ϵ_a than MSC approaches on average across all datasets. This validates the effectiveness of using time context as an input to handle asynchronous data.

We also note that for the OPPORTUNITY dataset, *Opp-MSC* has a higher ϵ_a than *Opp-SSC*. Without time context, the MSC cannot ignore stale packets, which can result in a higher error rate in the active region. We also note that the passive error rate (ϵ_p) is approximately the same across each approach since it represents misclassification when the classifier does *not* have updated information (effectively random guessing).

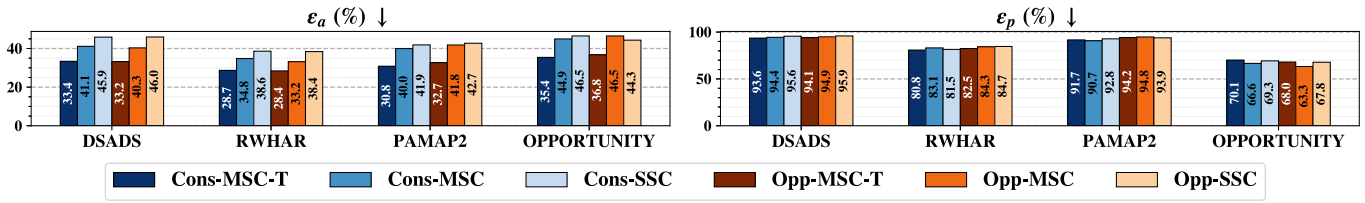


Fig. 8. Active (left) and passive (right) *error rates* for all policy-classifier combinations. For each policy group, the MSC-T approaches achieve a lower active error rate (ϵ_a) than the MSC approaches, which achieve a lower ϵ_a than the SSC approaches.

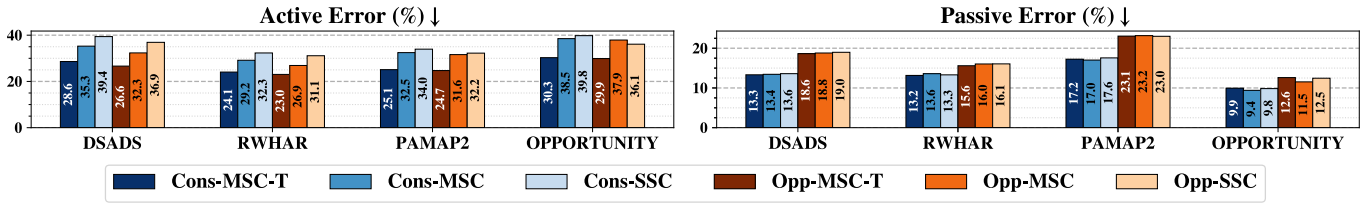


Fig. 9. Active (left) and passive (right) *errors* for all policy-classifier combinations. For each policy group, *active* error decreases from SSC to MSC-T. Conservative approaches (blue) have less passive error than opportunistic approaches (orange).

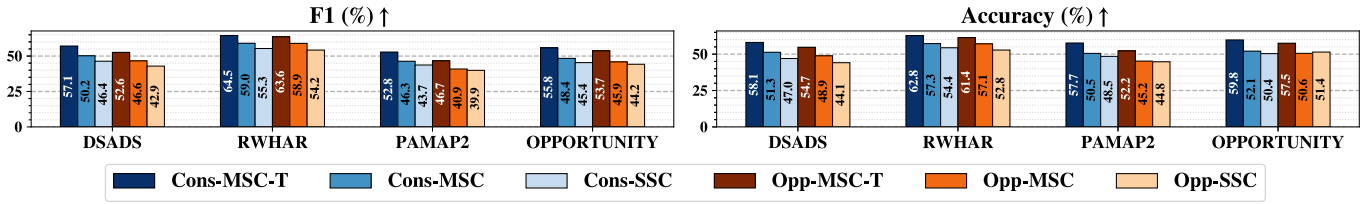


Fig. 10. $F1$-score (left) and accuracy (right) for all policy-classifier combinations. Across all datasets, we see that the Cons-MSC-T approach achieves the highest $F1$-score and accuracy.

3) Active/Passive Errors: In Fig. 9, we plot the active and passive errors, which add up to the total classification error as shown in (13). For each policy (color) group, *active* error decreases as we progress from SSC to MSC to MSC-T. Specifically, the MSC approaches achieve 2.2% less active error than the SSC approaches, while the MSC-T approaches achieve 6.5% less active error than the MSC approaches on average across all datasets. This matches the trend for ϵ_a and validates that minimizing ϵ_a reduces active error.

For the *passive* error, we see that the conservative approaches (blue) have a smaller passive error than the opportunistic approaches (orange). Specifically, the conservative approaches have 4% lower passive error compared to opportunistic approaches. This validates that reducing the probability of landing in the passive region (right side of Fig. 7) reduces passive error. Passive error is stable across classifiers since it depends on the policy. For a given classifier, active error slightly increases when going from an opportunistic to a conservative policy. This is expected since the goal of the conservative policy is to put *more* weight on ϵ_a by maximizing $P(k \in I_a)$ as discussed in the derivation of (14).

4) F1-Score/Accuracy: In Fig. 10, we plot the $F1$-score and accuracy. Across all datasets, *Cons-MSC-T* achieves the highest performance with a 7.5% increase in $F1$-score compared to *Cons-MSC*. This reflects the benefit of integrating time context. Further, *Cons-MSC-T* has a 3.5% higher $F1$-score compared to *Opp-MSC-T* (which only optimizes the classifier), highlighting the importance of optimizing the policy *in addition to* the classifier. This validates the benefit of optimizing *both* the sensor-level policies and downstream activity classifier.

C. Sensitivity to System Parameters

We test the sensitivity of our results to four system parameters to show that our approach maintains performance under harsher energy harvesting conditions. We show results for the *PAMAP2* dataset and `tinyhar` architecture.

1) Activity Duration: In Section IV-B, we sample activity durations from 10 to 30 s. Here we consider 0–10 and 30–40 s segments. In Fig. 11(a), we see that *Cons-MSC-T* consistently improves over the *Opp-SSC* baseline across activity durations. In general, the passive region increases with shorter activities as there is less time to harvest energy.

2) Energy Threshold E^{Th}: In Table II, we use a threshold of 50 μJ. Here, we also consider 25 and 100 μJ. In Fig. 11(b), we see that *Cons-MSC-T* maintains a consistent improvement over the *Opp-SSC* baseline across different threshold values. In general, the passive region increases with a larger threshold as it takes longer to harvest sufficient energy to acquire a packet.

3) Idle + Policy Energy E^{L}: In Table II, the energy consumption for the policy and idle state E^{L} is 0.264 μJ/sample (6.6 μW). Here, we also consider 3.3 and 13.2 μW. In Fig. 11(c), we see that *Cons-MSC-T* maintains a consistent improvement over the *Opp-SSC* baseline across different

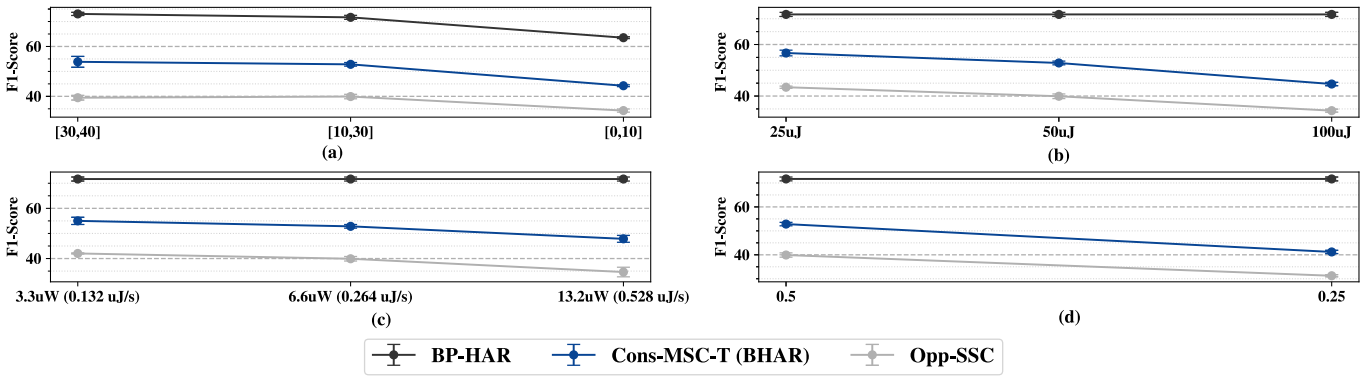


Fig. 11. Sensitivity to four system parameters: (a) altering the activity duration, (b) altering the energy threshold for data acquisition, (c) altering the energy consumption for the policy and idle state, and (d) altering the energy harvesting efficiency. Each line shows $F1$-score across the range of system parameter values for the PAMAP2 dataset and `tinyhar` architecture.

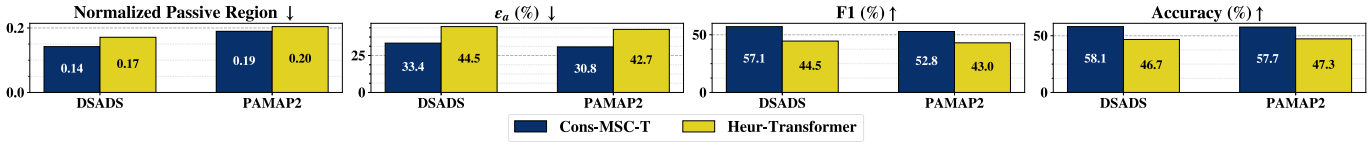


Fig. 12. Comparing Cons-MSC-T (blue) to the approach from [42] (yellow) on the DSADS and PAMAP2 datasets. We consider the passive region and active error rate to evaluate the policy and classifier, respectively.

values of E^{L}. In general, the passive region increases with larger E^{L} as it takes longer to harvest sufficient energy to acquire a packet, and the energy level will decay faster toward zero when the harvesting rate is low ($E^{\mathrm{H}} < E^{\mathrm{L}}$).

4) Energy Harvesting Efficiency: In all our experiments, we use a harvesting efficiency of 0.5. Here, we also consider 0.25, which corresponds to two times less energy harvested over the entire energy harvesting trace. In Fig. 11(d), we see that *Cons-MSC-T* maintains a consistent improvement over the *Opp-SSC* baseline under this more pessimistic energy harvesting state. This helps account for discrepancies between the energy harvesting model and prototype (Fig. 5).

D. Comparing to a Heuristic

To further validate BHAR, we compare *Cons-MSC-T* to the approach from [42], which uses a heuristic policy and a transformer-based classifier for sparse, asynchronous data. The policy acquires a packet whenever energy *accumulates* by αE^{Th} (where $\alpha \in [1, 2]$) or if no data is acquired after T seconds. This makes the policy conservative as it accumulates an energy surplus over time. Compared to π^{θ}, this policy is more rigid since α and T are fixed parameters that are the same across all sensors. We use a grid search to set α and T.

The classifier in [42] is designed as follows: a convolutional neural network (CNN) is used for each body part to convert accelerometer packets to embeddings. The packet arrival time and the body part a packet came from are then used to learn positional embeddings which provide spatial and temporal context relative to other packets. The embeddings are then fed into a two-layer transformer, global average pooling, and a fully connected layer for classification.

We plot the normalized passive region, active error rate ϵ_a, $F1$-score, and accuracy for the DSADS and PAMAP2 datasets in Fig. 12. We see that π^{θ} achieves a 3% lower passive region for DSADS but only a 1% lower passive region for PAMAP2 compared to the heuristic policy. This suggests that depending on the activity types, a fixed heuristic can be competitive with a learned policy. Intuitively, π^{θ} allows for different sensors to be more or less conservative, which may be more effective if different sensors have very different energy harvesting rates.

For the classifier, we see that the transformer-based model gets an ϵ_a that is more than 10% higher. This reflects the benefit of fine-tuning pretrained HAR models over training a domain-specific architecture from scratch.

E. Limitations and Future Work

1) Evaluation Trade-Offs: In our experiments, we estimate the energy harvested from existing HAR benchmarks and construct the activity sequences by concatenating activity segments. While this limits the realism of the data, it provides a mechanism to comprehensively compare classifiers and policies in multiple settings under identical energy harvesting conditions. In addition, it allows us to compare batteryless and BP approaches in a unified evaluation framework.

2) Future Work: In future work, we will conduct a comprehensive study with our hardware prototype with multiple human subjects. In addition, here we considered a simple, interpretable policy which can be optimized using a zero-order approach. More complex policies (e.g., neural networks) and optimization approaches (e.g., reinforcement learning) may improve performance at the cost of policy overhead. While we focus on HAR, the formulation of joint policy-classifier optimization for sparse, asynchronous sensing extends to other

time-series tasks where sensing and inference are tightly coupled. The decomposition of total error into active and passive components provides a task-agnostic lens for reasoning about performance when data availability is intermittent. Thus, we plan to leverage our framework for new tasks beyond HAR as well as other energy harvesting modalities.

VI. Conclusion

In this work, we unify BP and batteryless HAR under a common mathematical and evaluation framework. As the main theoretical contribution, we decompose the classification error into passive and active components, and develop BHAR, an optimization method for data acquisition and classification that targets both error types. While we achieve empirical improvements across multiple datasets and architectures, our primary contribution lies in setting the groundwork for future research: by formalizing the sources of error in batteryless HAR, we provide a foundation for researchers to design new classifiers and policies that bridge the performance gap in this emerging paradigm of self-powered sensing.

Acknowledgment

ChatGPT was used to improve grammar and clarity.

References

[1] S. Ahmed et al., "The Internet of Batteryless Things," *Commun. ACM*, vol. 67, no. 3, pp. 64–73, 2024.

[2] A. Bakar et al., "Protean: An energy-efficient and heterogeneous platform for adaptive and hardware-accelerated battery-free computing," in *Proc. 20th ACM Conf. Embedded Networked Sensor Syst.*, Nov. 2022, pp. 207–221.

[3] A. Curtiss et al., "FaceBit: Smart face masks platform," *Proc. ACM Interact., Mobile, Wearable Ubiquitous Technol.*, vol. 5, no. 4, pp. 1–44, Dec. 2021.

[4] J. Hester and J. Sorber, "The future of sensing is batteryless, intermittent, and awesome," in *Proc. 15th ACM Conf. Embedded Netw. Sensor Syst.*, Nov. 2017, pp. 1–6.

[5] A. Abedin, M. Ehsanpour, Q. Shi, H. Rezatofighi, and D. C. Ranasinghe, "Attend and discriminate: Beyond the state-of-the-art for human activity recognition using wearable sensors," *Proc. ACM Interact., Mobile, Wearable Ubiquitous Technol.*, vol. 5, no. 1, pp. 1–22, Mar. 2021.

[6] F. Ordóñez and D. Roggen, "Deep convolutional and LSTM recurrent neural networks for multimodal wearable activity recognition," *Sensors*, vol. 16, no. 1, p. 115, Jan. 2016.

[7] Y. Zhou, H. Zhao, Y. Huang, T. Riedel, M. Hefenbrock, and M. Beigl, "TinyHAR: A lightweight deep learning model designed for human activity recognition," in *Proc. ACM Int. Symp. Wearable Comput.*, Sep. 2022, pp. 89–93.

[8] A. Abedin, S. H. Rezatofighi, Q. Shi, and D. C. Ranasinghe, "SparseSense: Human activity recognition from highly sparse sensor data-streams using set-based neural networks," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, Aug. 2019, pp. 5780–5786.

[9] L. Xiao, Y. Meng, X. Tian, and H. Luo, "Energy allocation for activity recognition in wearable devices with kinetic energy harvesting," *Softw., Pract. Exper.*, vol. 51, no. 11, pp. 2185–2202, Nov. 2021.

[10] M. Deumer et al., "A battery-free wearable system for on-device human activity recognition using kinetic energy harvesting," in *Proc. Int. Conf. Embedded Wireless Syst. Netw.*, 2023, pp. 163–169.

[11] G. Cooper and R. Marculescu, "Packet pruning: Finding better energy spending policies for batteryless human activity recognition," in *Proc. IEEE 20th Int. Conf. Body Sensor Netw. (BSN)*, Oct. 2024, pp. 1–4.

[12] K. Chen, D. Zhang, L. Yao, B. Guo, Z. Yu, and Y. Liu, "Deep learning for sensor-based human activity recognition: Overview, challenges, and opportunities," *ACM Comput. Surv.*, vol. 54, no. 4, pp. 1–40, May 2022.

[13] J. Park, W.-S. Lim, D.-W. Kim, and J. Lee, "GTSNet: Flexible architecture under budget constraint for real-time human activity recognition from wearable sensor," *Eng. Appl. Artif. Intell.*, vol. 124, Sep. 2023, Art. no. 106543.

[14] J. Park, D.-W. Kim, and J. Lee, "CalaNet: Cheap all-layer aggregation for human activity recognition," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 37, 2024, pp. 69419–69444.

[15] Q. Teng, Y. Tang, and G. Hu, "RepHAR: Decoupling networks with accuracy-speed tradeoff for sensor-based human activity recognition," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–11, 2023.

[16] Z. Chen, L. Teng, L. Xu, J. Yu, and J. Liang, "MP-HAR: A novel motion-powered real-time human activity recognition system," *IEEE Internet Things J.*, vol. 11, no. 5, pp. 7652–7663, Mar. 2024.

[17] P. Mayer, M. Magno, and L. Benini, "Energy-positive activity recognition-from kinetic energy harvesting to smart self-sustainable wearable devices," *IEEE Trans. Biomed. Circuits Syst.*, vol. 15, no. 5, pp. 926–937, Oct. 2021.

[18] M. Buettner, R. Prasad, M. Philipose, and D. Wetherall, "Recognizing daily activities with RFID-based sensors," in *Proc. 11th Int. Conf. Ubiquitous Comput.*, Sep. 2009, pp. 51–60.

[19] P. Farina et al., "Memory-efficient energy-adaptive inference of pre-trained models on batteryless embedded systems," 2024, *arXiv:2405.10426*.

[20] G. Gobieski, B. Lucia, and N. Beckmann, "Intelligence beyond the edge: Inference on intermittent embedded systems," in *Proc. 24th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, Apr. 2019, pp. 199–213.

[21] B. Islam and S. Nirjon, "Zygarde: Time-sensitive on-device deep inference and adaptation on intermittently-powered systems," *Proc. ACM Interact., Mobile, Wearable Ubiquitous Technol.*, vol. 4, no. 3, pp. 1–29, Sep. 2020.

[22] R. L. S. Torres, D. C. Ranasinghe, Q. Shi, and A. P. Sample, "Sensor enabled wearable RFID technology for mitigating the risk of falls near beds," in *Proc. IEEE Int. Conf. RFID (RFID)*, Apr. 2013, pp. 191–198.

[23] A. Wickramasinghe and D. C. Ranasinghe, "Recognising activities in real time using body Worn passive sensors with sparse data streams: To interpolate or not to interpolate?" in *Proc. ICST*, 2016, pp. 21–30.

[24] M. M. Hassan, S. Ullah, M. S. Hossain, and A. Alelaiwi, "An end-to-end deep learning model for human activity recognition from highly sparse body sensor data in Internet of Medical Things environment," *J. Supercomput.*, vol. 77, no. 3, pp. 2237–2250, Mar. 2021.

[25] J. Grabocka, A. Nanopoulos, and L. Schmidt-Thieme, "Classification of sparse time series via supervised matrix factorization," in *Proc. AAAI Conf. Artif. Intell.*, 2012, vol. 26, no. 1, pp. 929–934.

[26] Y. Rubanova, R. T. Chen, and D. K. Duvenaud, "Latent ordinary differential equations for irregularly-sampled time series," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 5320–5330.

[27] S. N. Shukla and B. Marlin, "Multi-time attention networks for irregularly sampled time series," in *Proc. Int. Conf. Learn. Represent.*, 2021, pp. 14897–14911.

[28] C. Martin-Turrero, M. Bouvier, M. Breitenstein, P. Zanuttigh, and V. Parret, "Alert-transformer: Bridging asynchronous and synchronous machine learning for real-time event-based spatio-temporal data," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024, pp. 48837–48854.

[29] N. Messikommer, D. Gehrig, A. Loquercio, and D. Scaramuzza, "Event-based asynchronous sparse convolutional networks," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 415–431.

[30] S. Tipirneni and C. K. Reddy, "Self-supervised transformer for sparse and irregularly sampled multivariate clinical time-series," *ACM Trans. Knowl. Discovery Data*, vol. 16, no. 6, pp. 1–17, Dec. 2022.

[31] J. Zhang, S. Zheng, W. Cao, J. Bian, and J. Li, "Warpformer: A multi-scale modeling approach for irregular clinical time series," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2023, pp. 3273–3285.

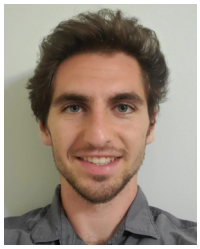
[32] Y. Tuncel, G. Bhat, J. Park, and U. Y. Ogras, "ECO: Enabling energy-neutral IoT devices through runtime allocation of harvested energy," *IEEE Internet Things J.*, vol. 9, no. 7, pp. 4833–4848, Apr. 2022.

[33] N. Yamin and G. Bhat, "Uncertainty-aware energy harvest prediction and management for IoT devices," *ACM Trans. Design Autom. Electron. Syst.*, vol. 28, no. 5, pp. 1–33, Sep. 2023.

[34] N. Yamin and G. Bhat, "Indoor–outdoor energy management for wearable IoT devices with conformal prediction and rollout," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 11, pp. 3370–3381, Nov. 2024.

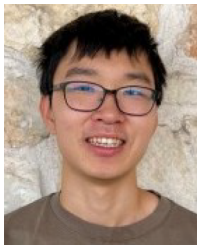
[35] D. Hussein, N. Yamin, and G. Bhat, "EMI: Energy management meets imputation in wearable IoT devices," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 11, pp. 3792–3803, Nov. 2024.

[36] T. Basaklar, Y. Tuncel, and U. Y. Ogras, "TinyMAN: Lightweight energy manager using reinforcement learning for energy harvesting wearable IoT devices," 2022, *arXiv:2202.09297*.
[37] T. Basaklar, Y. Tuncel, S. Gumussoy, and U. Ogras, "GEM-RL: Generalized energy management of wearable devices using reinforcement learning," in *Proc. Design, Autom. Test Eur. Conf. Exhib.* (DATE), Apr. 2023, pp. 1–6.
[38] B. Lucia, V. Balaji, A. Colin, K. Maeng, and E. Ruppel, "Intermittent computing: Challenges and opportunities," in *Proc. 2nd Summit Adv. Program. Lang.*, 2017, pp. 1–8.
[39] Y. Chang, A. Mathur, A. Isopoussu, J. Song, and F. Kawsar, "A systematic study of unsupervised domain adaptation for robust human-activity recognition," *Proc. ACM Interact., Mobile, Wearable Ubiquitous Technol.*, vol. 4, no. 1, pp. 1–30, Mar. 2020.
[40] H. Haresamudram, I. Essa, and T. Plötz, "Assessing the state of self-supervised human activity recognition using wearables," *Proc. ACM Interact., Mobile, Wearable Ubiquitous Technol.*, vol. 6, no. 3, pp. 1–47, Sep. 2022.
[41] Y. Zhou, T. King, H. Zhao, Y. Huang, T. Riedel, and M. Beigl, "MLP-HAR: Boosting performance and efficiency of HAR models on edge devices with purely fully connected layers," in *Proc. ACM Int. Symp. Wearable Comput.*, Oct. 2024, pp. 133–139.
[42] G. Cooper, T. Huang, and R. Marculescu, "Demo abstract: A prototype for machine learning with batteryless sensors," in *Proc. IEEE/ACM 9th Int. Conf. Internet-Things Design Implement. (IoTDI)*, May 2024, pp. 223–224.
[43] Z. Che, S. Purushotham, K. Cho, D. Sontag, and Y. Liu, "Recurrent neural networks for multivariate time series with missing values," *Sci. Rep.*, vol. 8, no. 1, p. 6085, Apr. 2018.
[44] R. Hooke and T. A. Jeeves, "'Direct search' solution of numerical and statistical problems," *J. ACM*, vol. 8, no. 2, pp. 212–229, 1961.
[45] J. Nocedal and S. J. Wright, *Numerical Optimization*. Cham, Switzerland: Springer, 2006.
[46] K. Altun, B. Barshan, and O. Tunçel, "Comparative study on classifying human activities with miniature inertial and magnetic sensors," *Pattern Recognit.*, vol. 43, no. 10, pp. 3605–3620, Oct. 2010.
[47] T. Sztyler and H. Stuckenschmidt, "On-body localization of wearable devices: An investigation of position-aware activity recognition," in *Proc. IEEE Int. Conf. Pervasive Comput. Commun. (PerCom)*, Mar. 2016, pp. 1–9.
[48] A. Reiss and D. Stricker, "Introducing a new benchmarked dataset for activity monitoring," in *Proc. 16th Int. Symp. Wearable Comput.*, Jun. 2012, pp. 108–109.
[49] D. Roggen et al., "Collecting complex activity datasets in highly rich networked sensor environments," in *Proc. 7th Int. Conf. Networked Sens. Syst. (INSS)*, Jun. 2010, pp. 233–240.
[50] X. Liu, Q. Zhou, C.-T. Cheng, J. Wu, X. Wang, and L. Fan, "An ω-free gyro-free accelerometer pair algorithm for 2D trajectory reconstruction," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 16173–16177, Dec. 2020.
[51] S. Khalifa, M. Hassan, and A. Seneviratne, "Pervasive self-powered human activity recognition without the accelerometer," in *Proc. IEEE Int. Conf. Pervasive Comput. Commun. (PerCom)*, Mar. 2015, pp. 79–86.
[52] S. Khalifa, G. Lan, M. Hassan, A. Seneviratne, and S. K. Das, "HARKE: Human activity recognition from kinetic energy harvesting data in wearable devices," *IEEE Trans. Mobile Comput.*, vol. 17, no. 6, pp. 1353–1368, Jun. 2018.
[53] M. Gorlatova, J. Sarik, G. Grebla, M. Cong, I. Kymissis, and G. Zussman, "Movers and shakers: Kinetic energy harvesting for the Internet of Things," *IEEE J. Sel. Areas Commun.*, vol. 33, no. 8, pp. 1624–1639, Aug. 2015.
[54] Nordic Semiconductor. (2025). *NRF52DK*. [Online]. Available: https://www.nordicsemi.com/Products/Development-hardware/nRF52-DK
[55] (2025). Power Profiler Kit II. [Online]. Available: https://www.nordicsemi.com/Products/Development-hardware/Power-Profiler-Kit-2
[56] (2025). NRF52832. [Online]. Available: https://www.nordicsemi.com/Products/nRF52832
[57] Bosch Sensortec. (2025). BMA400. [Online]. Available: https://www.bosch-sensortec.com/products/motion-sensors/accelerometers/bma400/
[58] Analog Devices. (2025). LTC3588. [Online]. Available: https://www.analog.com/en/products/ltc3588-1.html



Geffen Cooper (Graduate Student Member, IEEE) received the B.S. degree in computer engineering from the University of California, Santa Barbara, Santa Barbara, CA, USA, in 2022. He is currently pursuing the Ph.D. degree in electrical and computer engineering with The University of Texas at Austin, Austin, TX, USA.

His research interests include machine learning with batteryless sensors, wearables, and edge AI.



Nathan Tsao received the B.S. degree in mechanical engineering from the University of Illinois Urbana–Champaign, Champaign, IL, USA, in 2022, and the M.S. degree in mechanical engineering from the University of Texas at Austin, Austin, TX, USA, in 2025.



Filippos Fotiadis (Member, IEEE) was born in Thessaloniki, Greece. He received the Diploma degree in electrical and computer engineering from the Aristotle University of Thessaloniki, Thessaloniki, in 2018, the dual M.S. degree in aerospace engineering and mathematics from Georgia Tech, Atlanta, GA, USA, in 2022 and 2023, respectively, and the Ph.D. degree in aerospace engineering from Georgia Tech in 2024.

He is currently a Post-Doctoral Researcher with the Oden Institute for Computational Engineering and Sciences, The University of Texas at Austin, Austin, TX, USA. His current research interests are in the intersection of control theory, game theory, and machine learning, with applications to the autonomy and security of cyber–physical and aerospace systems.



Ufuk Topcu (Fellow, IEEE) is currently a Professor with the Department of Aerospace Engineering and Engineering Mechanics, The University of Texas at Austin, Austin, TX, USA, where he holds the Temple Foundation Endowed Professorship No. 1. He is a core Faculty Member with the Oden Institute for Computational Engineering and Sciences and Texas Robotics, Austin, and the Director of the Autonomous Systems Group, Austin, USA. His research interests include theoretical and algorithmic aspects of the design and verification of autonomous systems, typically in the intersection of formal methods, reinforcement learning, and control theory



Radu Marculescu (Fellow, IEEE) received the Ph.D. degree in electrical engineering from the University of Southern California, Los Angeles, CA, USA, in 1998.

From 2000 to 2019, he was a Professor with the Electrical and Computer Engineering Department, Carnegie Mellon University, Pittsburgh, PA, USA. He is currently a Professor and the Laura Jennings Turner Chair of Engineering with the Department of Electrical and Computer Engineering, The University of Texas at Austin, Austin, TX, USA. His current research interests include developing artificial intelligence/machine learning algorithms for applications in computer vision, bioimaging, and the Internet of Things.